



GUESS THE NUMBER GAME

A Python-Based Interactive Number Guessing Game

PROJECT REPORT

STUDENT NAME : PRIYAM PARASHAR

Registration Number : 26BCE11516

Slot no. : A21+A22+A23+B14+B22+D21+D22

Under the guidance of:

I. JASMINE SELVAKUMARI JEYA

School of Computer Science and Engineering
VIT Bhopal University

SUBMISSION DATE :

30 September 2026

TABLE OF CONTENTS

1.	INTRODUCTION	1
2.	PROBLEM STATEMENT	2
3.	OBJECTIVES	3
4.	FUNCTIONAL REQUIREMENTS	4
5.	NON-FUNCTIONAL REQUIREMENTS	5
6.	SYSTEM ARCHITECTURE	6
7.	DESIGN DIAGRAMS	7
7.1	Use Case Diagram	7
7.2	Workflow Diagram	8
7.3	Sequence Diagram	9
7.4	Class Diagram	10
8.	DESIGN DECISIONS & RATIONALE	11
9.	IMPLEMENTATION DETAILS	12
9.1	Module: Main.py	12
9.2	Module: Menus.py	13
9.3	Module: Gameplay.py	14
9.4	Module: Database.py	15
9.5	Module: Assets.py	16
9.6	Module: Settings.py	17
9.7	Module: Achievements.py	18
10.	SCREENSHOTS	19
11.	TESTING APPROACH	25
12.	CHALLENGES FACED	27

13.	LEARNINGS & KEY TAKEAWAYS	28
14.	FUTURE ENHANCEMENTS	29
15.	REFERENCES	30
	APPENDIX SOURCE CODE	31

1. INTRODUCTION

Number guessing games have been a classic programming exercise for decades, serving as an excellent introduction to fundamental programming concepts such as random number generation, user input handling, conditional logic, and iterative processes. This project extends the basic concept into a fully-featured, production-ready game with multiple difficulty levels, persistent player statistics, achievement tracking, and competitive leaderboards. The "Guess The Number" game is built entirely in Python, leveraging the language's simplicity and powerful standard library to create an engaging user experience within a console-based interface. Despite the absence of graphical elements, the game employs creative ASCII art to provide visual feedback and enhance the overall gaming experience. The project demonstrates mastery of several key software engineering principles including modular design, separation of concerns, data persistence, error handling, and user experience design. Each module serves a distinct purpose, making the codebase maintainable, extensible, and easy to understand. This report documents the complete development lifecycle of the project, from initial problem identification through design decisions, implementation details, testing strategies, and lessons learned. It serves as both technical documentation and a reflection on the development process.

1.1 Project Overview

The game operates on a simple premise: the computer selects a random number within a predefined range, and the player must guess it within a limited number of attempts. After each incorrect guess, the game provides feedback indicating whether the guess was too high or too low, allowing the player to narrow down their search. What sets this implementation apart from basic tutorials is its comprehensive feature set:

- Six difficulty levels with progressively challenging parameters
- Dynamic scoring system with multipliers and penalties
- Achievement system with six unlockable badges
- Global leaderboard tracking all players' scores
- Individual player profiles with detailed statistics
- Hint system with strategic point deductions
- Persistent data storage across gaming sessions
- Motivational and demotivational feedback messages
- Custom game mode for user-defined parameters

2. PROBLEM STATEMENT

Traditional number guessing games found in programming tutorials are often minimal implementations that lack the depth and features expected in a modern gaming application. They typically feature:

- Single difficulty level with no variety
- No persistence of player data between sessions
- Absence of scoring or reward mechanisms
- Limited or no feedback beyond 'higher' or 'lower'
- No competitive elements like leaderboards
- Poor user experience with minimal visual feedback
- No tracking of player statistics or achievements

There exists a need for an educational yet engaging number guessing game that demonstrates advanced Python programming concepts while providing an entertaining experience with features that encourage repeated playthroughs and skill improvement. The game should serve as both a learning tool for understanding programming fundamentals and a complete gaming application with modern features.

2.1 Proposed Solution

The proposed solution addresses these limitations by implementing a comprehensive number guessing game with the following characteristics:

- Multiple difficulty tiers to accommodate players of all skill levels
- JSON-based data persistence for player statistics and achievements
- Point-based scoring system with multipliers for efficient guessing
- Hint system that provides strategic assistance at a cost
- Achievement badges that reward specific accomplishments
- Global leaderboard fostering competition among players
- Detailed player profiles tracking comprehensive statistics
- ASCII art visuals providing thematic feedback for each mode
- Dynamic motivational messages maintaining player engagement

3. OBJECTIVES

The primary objectives of this project are:

- To design and implement a fully functional number guessing game using Python programming concepts
- To demonstrate modular software design through separation of concerns across multiple files
- To implement persistent data storage for maintaining player statistics across gaming sessions
- To create an engaging user experience with visual feedback, achievements, and competitive elements
- To apply error handling and input validation for robust application behavior
- To showcase clean code practices with proper documentation and organization

3.1 Learning Objectives

From an educational perspective, this project aims to:

- Reinforce understanding of Python functions, modules, and imports
- Practice file I/O operations with JSON data serialization
- Implement random number generation and probability concepts
- Design user interfaces for console-based applications
- Apply data structures (dictionaries, lists) for state management
- Demonstrate exception handling for graceful error recovery
- Create maintainable code through modular architecture

4. FUNCTIONAL REQUIREMENTS

The functional requirements define the specific features and capabilities that the system must provide to users. These requirements were identified based on the need to create a comprehensive gaming experience.

4.1 FR1: User Management

The system shall allow players to:

- Enter their name at game startup
- Automatically create a new player profile on first entry
- Switch between different player profiles during gameplay
- View their personal statistics and achievements
- Maintain persistent identity across gaming sessions

4.2 FR2: Game Modes and Difficulty Selection

The system shall provide multiple difficulty levels:

Mode	Range	Attempts	Base Points
Easy	1-50	10	100
Medium	1-100	7	200
Hard	1-200	5	400
Extreme	1-500	5	1000
Nightmare	1-1000	4	2000
Custom	User-defined	User-defined	Calculated

4.3 FR3: Core Gameplay Mechanics

The system shall implement the following gameplay features:

- Generate a random number within the specified range
- Accept and validate user input for guesses
- Provide feedback on whether guesses are too high or too low
- Track guess history and display it to the player

- Enforce attempt limits and end game when exhausted
- Detect correct guesses and declare victory
- Calculate and award points based on performance

4.4 FR4: Hint System

The system shall provide a hint mechanism:

- Allow players to request up to 2 hints per game
- First hint reveals whether the number is even or odd
- Second hint reveals divisibility by a random number (3, 4, 5, 7, or 10)
- Apply a 25% point penalty per hint used
- Prevent hint requests when no hints remain

4.5 FR5: Scoring System

The system shall implement a dynamic scoring mechanism:

- Calculate base points based on difficulty level
- Apply multiplier based on remaining attempts: $(\text{max_attempts} - \text{attempts_used} + 1)$
- Deduct 25% of earned points for each hint used
- Accumulate total score across all gaming sessions
- Display earned points immediately after each victory

4.6 FR6: Achievement System

The system shall track and reward player accomplishments:

Achievement	Unlock Condition
First Blood	Win your very first game
Veteran	Play 25 total games
High Roller	Accumulate over 5,000 total points
Flawless Victory	Guess the number on your very first try
Nightmare Slayer	Win a game on Nightmare difficulty
No Help Needed	Win a Hard or higher game without hints

4.7 FR7: Leaderboard

The system shall display a global leaderboard showing:

- All registered players ranked by total score
- Player name (truncated if exceeding 15 characters)
- Total accumulated score
- Total number of wins
- Win percentage calculated from games played

4.8 FR8: Player Profile

The system shall provide individual player profiles displaying:

- Total points accumulated across all sessions
- Total games played
- Total wins achieved
- Win rate percentage
- Personal best attempts for each difficulty mode
- List of unlocked achievements with descriptions

5. NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define the quality attributes and constraints that the system must satisfy. These requirements ensure the application is robust, maintainable, and provides a good user experience.

5.1 NFR1: Performance

The system shall:

- Load and save data files within 100 milliseconds
- Respond to user input within 50 milliseconds
- Handle up to 1000 unique player profiles without degradation
- Maintain game state in memory for instant access during gameplay

5.2 NFR2: Usability

The system shall:

- Provide clear, intuitive menu navigation
- Display helpful error messages for invalid input
- Use consistent formatting throughout all screens
- Include visual ASCII art to enhance engagement
- Offer contextual help through hint system
- Support case-insensitive input where appropriate

5.3 NFR3: Reliability

The system shall:

- Handle corrupted save files gracefully by starting fresh
- Prevent data loss through immediate save after each game
- Validate all user input before processing
- Catch and handle exceptions to prevent crashes
- Support graceful shutdown via keyboard interrupt (Ctrl+C)

5.4 NFR4: Maintainability

The system shall:

- Use modular architecture with single-responsibility modules
- Include comments and docstrings for all functions
- Follow consistent naming conventions (snake_case for functions/variables)
- Centralize configuration in Settings.py for easy modification
- Separate data (Assets.py, Settings.py) from logic (Gameplay.py, Menus.py)

5.5 NFR5: Scalability

The system shall:

- Support unlimited number of player profiles
- Allow addition of new difficulty modes without code restructuring
- Enable easy extension of achievement types
- Accommodate future features like multiplayer or networked leaderboards

5.6 NFR6: Error Handling

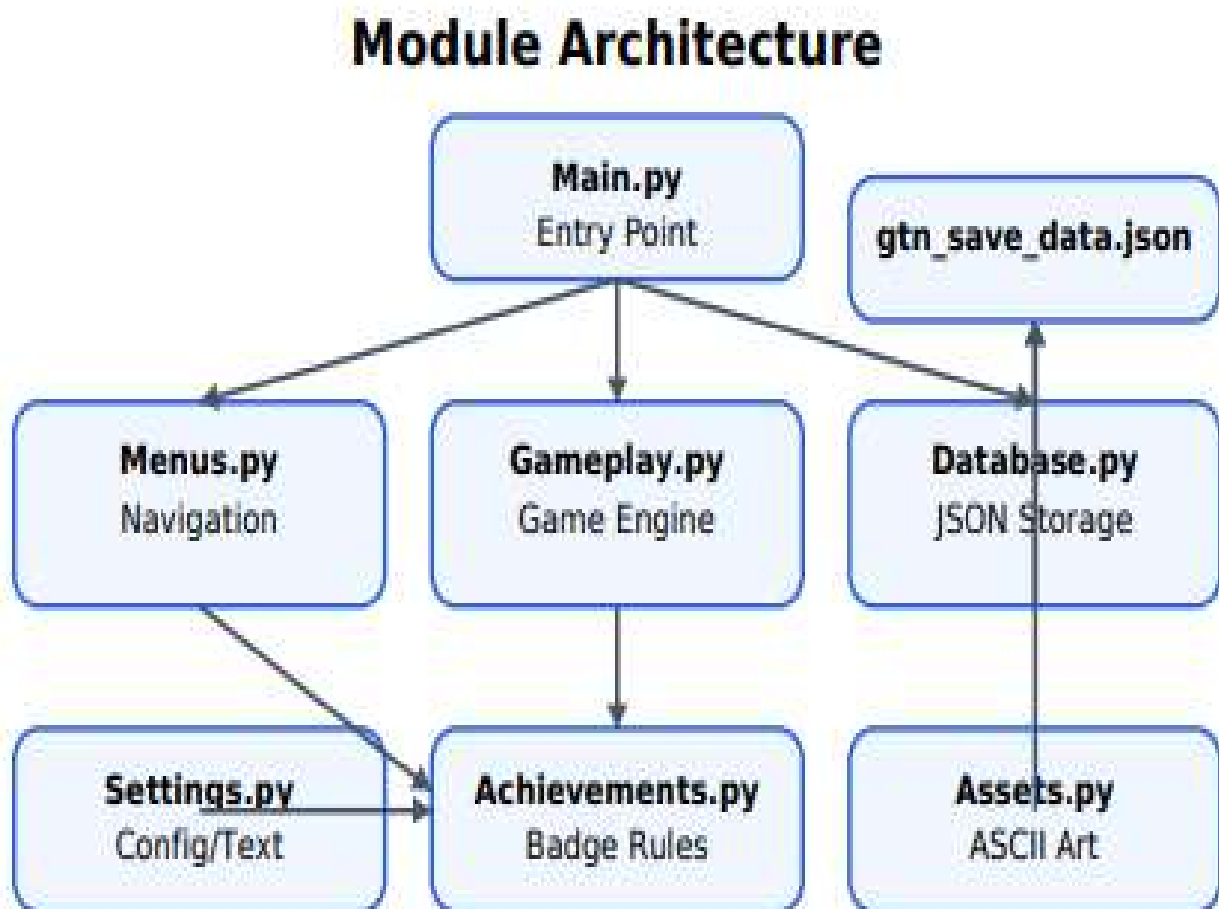
The system shall:

- Catch JSON decode errors and provide fallback behavior
- Validate numeric input and prompt for retry on failure
- Prevent duplicate guesses without penalizing the player
- Handle file system errors during save/load operations
- Display user-friendly messages instead of stack traces

6. SYSTEM ARCHITECTURE

The system follows a modular, layered architecture that separates concerns across seven distinct Python modules. This architecture promotes code reusability, maintainability, and clear separation between user interface, business logic, and data persistence layers.

6.1 High-Level Architecture Diagram



6.2 Module Descriptions

6.2.1 Main.py (Entry Point)

Serves as the application's entry point. Initializes the database, displays the title screen, captures player name, and runs the main game loop that presents the primary menu and routes user choices to appropriate handlers.

6.2.2 Menus.py (User Interface Layer)

Implements all user-facing menus including difficulty selection, leaderboard display, and player profile viewing. Handles user input validation and navigation between different screens. Acts as the intermediary between user actions and game logic.

6.2.3 Gameplay.py (Business Logic Layer)

Contains the core game mechanics including random number generation, guess validation, hint provision, scoring calculation, and win/loss determination. This module implements the primary game loop for individual rounds and orchestrates calls to the Achievements and Database modules.

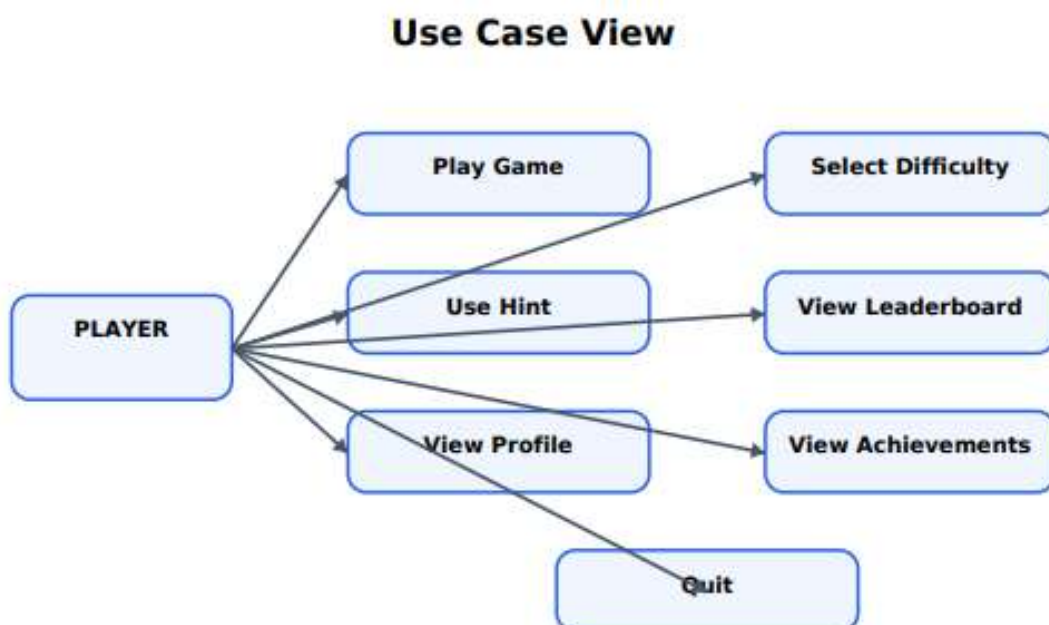
6.2.4 Database.py (Data Persistence Layer)

Manages all data persistence operations including loading player data from JSON files, saving updated statistics, and initializing new player profiles. Handles file I/O operations and provides data migration for legacy save files.

7. DESIGN DIAGRAMS

7.1 Use Case Diagram

The Use Case Diagram illustrates the interactions between players and the system, showing all major functionalities available to users.



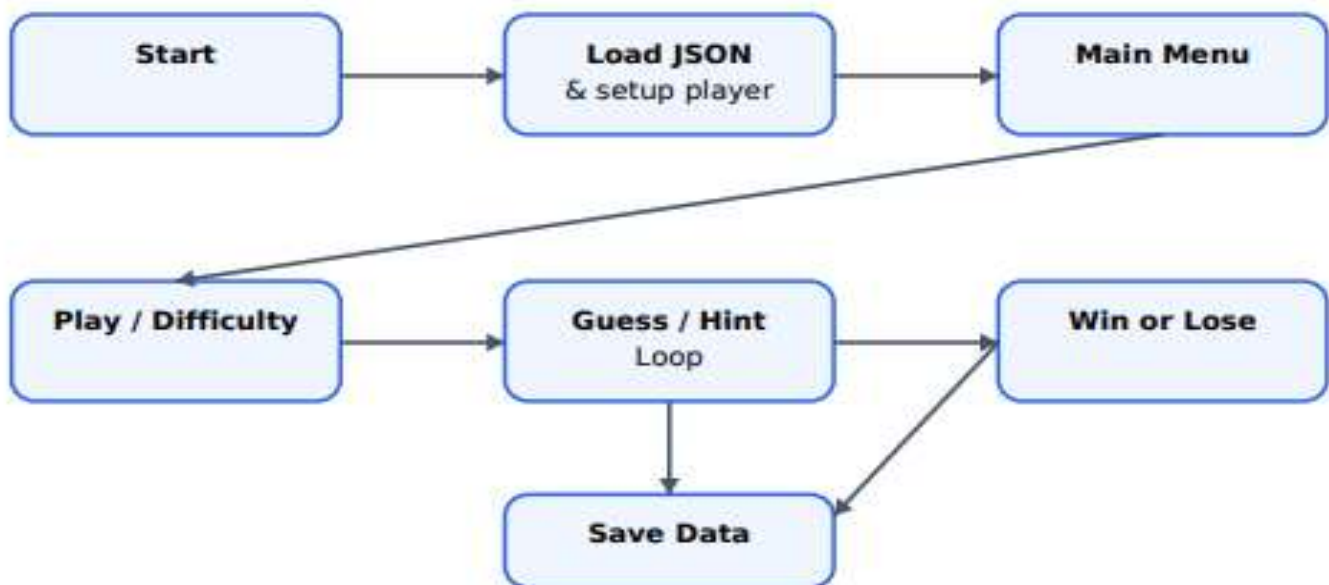
Use Case Descriptions:

- Start Game: Player enters name and initializes session
- Select Difficulty: Player chooses from 6 difficulty modes
- Make Guess: Player inputs number guesses during gameplay
- Request Hint: Player asks for hints (max 2 per game)
- View Leaderboard: Player views global rankings
- View Profile: Player examines personal statistics and achievements
- Switch Player: Player changes active profile
- Quit Game: Player exits application gracefully

7.2 Workflow Diagram

The Workflow Diagram shows the complete flow of the application from startup to shutdown, including all decision points and loops.

Player Workflow

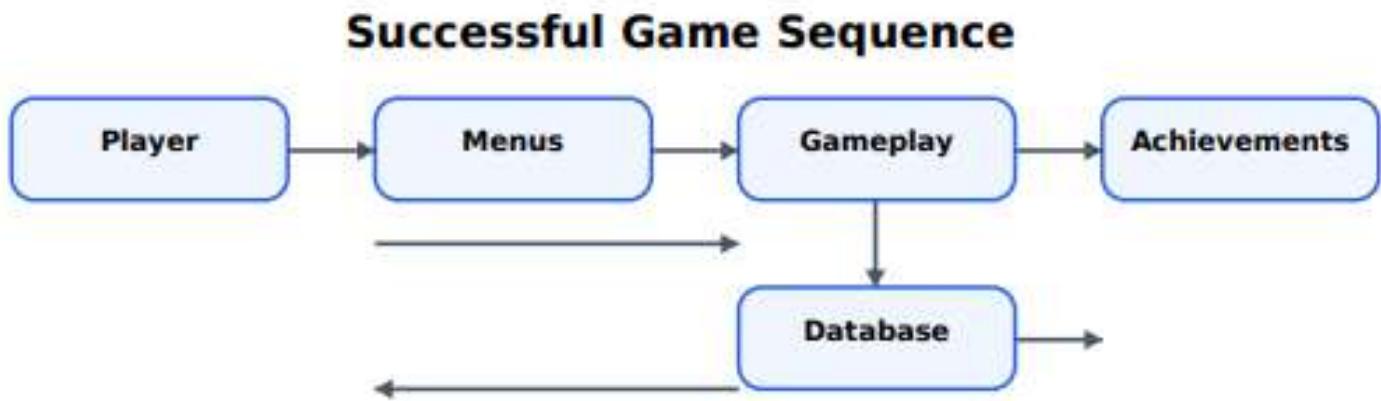


Key Workflow Stages:

- Initialization: Load database, display title, capture player name
- Main Loop: Present menu, process user choice, route to handler
- Game Round: Generate number, process guesses, provide feedback
- Resolution: Determine win/loss, calculate score, update statistics
- Persistence: Save data to JSON file, return to menu
- Termination: Display farewell message, exit gracefully

7.3 Sequence Diagram

The Sequence Diagram depicts the temporal interactions between modules during a typical game round, showing the order of method calls and data exchanges.



Key Interactions:

- Player - Main: Enter name
- Main - Database: setup_player(db, name)
- Database - Main: Return updated db
- Player - Menus: Select difficulty option
- Menus - Gameplay: run_round(params)
- Gameplay - Database: Load player stats
- Gameplay - Gameplay: Generate random number
- Player - Gameplay: Submit guess
- Gameplay - Player: Provide feedback (high/low/hint)
- Gameplay - Achievements: evaluate_achievements()
- Achievements - Settings: Lookup achievement details
- Gameplay - Database: save_game_data(db)
- Gameplay - Menus: Return to menu

8. DESIGN DECISIONS & RATIONALE

Several key design decisions were made during development to ensure the project meets its objectives while maintaining code quality and user experience. This section documents those decisions and the reasoning behind them.

8.1 Decision: Modular Architecture

Choice: Split the application into seven separate Python files rather than a single monolithic script.

- Improves code organization and readability
- Enables focused debugging and testing of individual modules
- Facilitates future extensions without disrupting existing code
- Demonstrates professional software engineering practices
- Makes it easier to understand data flow and dependencies

8.2 Decision: JSON for Data Persistence

Choice: Use JSON files instead of databases, CSV, or pickle for storing player data.

- Human-readable format allows manual inspection and debugging
- Built-in Python support via json module (no external dependencies)
- Cross-platform compatibility
- Simple structure matches the hierarchical nature of player data
- Easy to parse and modify programmatically

8.3 Decision: ASCII Art for Visuals

Choice: Use ASCII art instead of graphical libraries like Pygame or Tkinter.

- No external dependencies beyond Python standard library
- Works in any terminal/console environment
- Demonstrates creativity within constraints
- Keeps focus on programming concepts rather than graphics programming
- Easier to version control and share as text files

8.4 Decision: Immediate Save After Each Game

Choice: Save player data to disk immediately after each game round rather than batching saves or saving only on exit.

- Prevents data loss if application crashes or is force-closed
- Ensures achievements and statistics are always up to date
- Simple implementation without complex state management
- Performance impact is negligible for small JSON files

8.5 Decision: Dynamic Scoring with Multipliers

Choice: Implement a scoring system that multiplies base points by remaining attempts rather than flat points per game.

- Rewards skillful play (fewer attempts = higher score)
- Creates strategic depth (risk vs. reward for hints)
- Encourages replayability to beat personal bests
- Differentiates performance across difficulty levels
- Provides meaningful leaderboard competition

8.6 Decision: Two-Tier Hint System

Choice: Provide exactly two hints with escalating information (parity, then divisibility) at a 25% point penalty each.

- Prevents over-reliance on hints while still offering assistance
- Parity hint provides basic narrowing without giving away the answer
- Divisibility hint offers more strategic information
- Point penalty creates meaningful trade-off decision
- Two hints balance assistance with challenge

8.7 Decision: Motivational and Demotivational Messages

Choice: Include randomized feedback messages that can be either encouraging or teasing based on player performance.

- Adds personality and humor to the gaming experience
- Demotivational messages provide comic relief without real penalty
- Motivational messages encourage persistence after failures
- Randomization keeps feedback fresh across multiple playthroughs
- Centralized in Settings.py for easy customization

9. IMPLEMENTATION DETAILS

This section provides a detailed walkthrough of each module's implementation, including key functions, algorithms, and code snippets that demonstrate the application of programming concepts.

9.1 Module: Main.py

The main.py file serves as the application's entry point and orchestrates the overall game flow.

Key Implementation Points:

- Imports Assets, Database, and Menus modules
- Defines main() function as the primary entry point
- Loads game data from JSON file at startup
- Displays title ASCII art using Assets.TITLE_ART
- Captures player name with input validation (defaults to 'Guest' if blank)
- Calls Database.setup_player() to initialize or update player profile
- Enters infinite while loop for main menu
- Presents four options: Play, Leaderboard, Profile, Quit
- Routes user choice to appropriate Menus function
- Handles KeyboardInterrupt for graceful shutdown

Code Snippet (Main Loop):

```
def main(): db = Database.load_game_data() print(Assets.TITLE_ART) current_player = input("Enter your name to start: ").strip().capitalize() if not current_player: current_player = "Guest" db = Database.setup_player(db, current_player) Database.save_game_data(db) while True: print("\n--- MAIN MENU ---") print("1. Play Game\n2. Leaderboard\n3. My Profile\n4. Quit") choice = input("> ").strip() if choice == '1': Menus.difficulty_select(current_player, db) elif choice == '2': Menus.display_leaderboard(db) elif choice == '3': current_player = Menus.view_profile(current_player, db) elif choice == '4': print(Assets.QUIT_ART); break
```

9.2 Module: Menus.py

The menus.py file implements all user interface screens and navigation logic.

9.2.1 difficulty_select(player_name, db)

- Displays six difficulty options plus Custom and Back
- Maps each choice to specific range, attempts, and ASCII art
- Calls Gameplay.run_round() with appropriate parameters
- Handles custom mode by prompting for user-defined range and attempts
- Validates custom input (high ≥ 2 , tries ≥ 1)
- Catches ValueError for non-numeric custom input

9.2.2 display_leaderboard(db)

- Displays ASCII art header for leaderboard
- Checks if database is empty and provides feedback
- Sorts players by total_score in descending order
- Formats output as a table with Rank, Player, Score, Wins, Win %
- Truncates player names exceeding 15 characters
- Calculates win rate as $(\text{wins} / \text{games_played} * 100)$
- Handles division by zero for players with no games

9.2.3 view_profile(player_name, db)

- Displays player profile menu with four options
- View Stats: Shows total points, games, wins, win rate, personal bests
- View Badges: Lists all unlocked achievements with titles and descriptions
- Switch Player: Allows changing active player profile
- Back: Returns to main menu
- Personal bests display 'Unbeaten' for modes never completed

9.3 Module: Gameplay.py

The gameplay.py file contains the core game logic and is the most complex module in the application.

Key Function: `run_round(player_name, db, upper_bound, max_attempts, mode_name, art_asset)`

Game Flow:

- Step 1: Display ASCII art and mode header
- Step 2: Generate secret number using `random.randint(1, upper_bound)`
- Step 3: Initialize attempts counter, guesses list, and hints tracking
- Step 4: Calculate base points as `100 * (upper_bound // 50)`
- Step 5: Enter game loop (while attempts < max_attempts)
- Step 6: Display guess history if guesses exist
- Step 7: Capture and sanitize user input (strip, lowercase)
- Step 8: Handle 'hint' command with validation and penalty logic
- Step 9: Validate numeric input and catch `ValueError`
- Step 10: Check for duplicate guesses and skip without penalty
- Step 11: Increment attempts and append to guesses list
- Step 12: Compare guess to secret number

Win Condition Logic:

- If `guess == secret`: Display `WIN_ART`, calculate score
- Score formula: `base_pts * (max_attempts - attempts + 1)`
- Apply hint penalty: `earned -= int(earned * 0.25 * hints_used)`
- Update player stats: `games_played++`, `wins++`, `total_score += earned`
- Check for new personal best in current mode
- Call `Achievements.evaluate_achievements()`
- Save data to JSON file
- Pause for user acknowledgment before returning

Loss Condition Logic:

- If `attempts == max_attempts` and no correct guess: Display `LOSE_ART`
- Reveal the secret number
- Increment `games_played` counter (no win)
- Save data to JSON file
- Pause for user acknowledgment before returning

9.4 Module: Database.py

The database.py file handles all data persistence operations using JSON files.

9.4.1 load_game_data()

- Checks if Settings.SAVE_FILE exists using os.path.exists()
- If exists: Opens file in read mode and parses JSON
- Catches json.JSONDecodeError for corrupted files
- Returns empty dict {} if file doesn't exist or is corrupted
- Prints warning message if starting fresh due to corruption

9.4.2 save_game_data(db)

- Opens Settings.SAVE_FILE in write mode
- Serializes db dictionary to JSON with indent=4 for readability
- Automatically creates file if it doesn't exist
- Overwrites previous data (full save, not incremental)

9.4.3 setup_player(db, name)

- Checks if player name exists in database
- If new player: Creates profile with default values (0 games, 0 wins, 0 score, empty achievements, null best_scores)
- If existing player: Patches old profiles to include new fields (achievements, best_scores, total_score)
- Uses .get() method for safe access to potentially missing keys
- Returns updated db dictionary

9.5 Module: Assets.py

The assets.py file stores all ASCII art as raw strings to avoid escaping issues with backslashes.

ASCII Art Collection:

- TITLE_ART: Game title banner with 'Can you beat the machine?' tagline
- WIN_ART: Trophy/cup symbol for victory
- LOSE_ART: Sad face with 'GAME OVER' text
- EASY_ART: Smiling face for easy mode
- MEDIUM_ART: Neutral face for medium mode
- HARD_ART: Sword/weapon for hard mode
- EXTREME_ART: Monster/creature for extreme mode
- NIGHTMARE_ART: Dragon/demon for nightmare mode
- CUSTOM_ART: User-defined/custom game icon
- LEADERBOARD_ART: Trophy podium for leaderboard
- PROFILE_ART: Player card/profile icon
- QUIT_ART: RIP tombstone for game over/quit

Implementation Note:

All strings use raw string literals (r"...") to preserve backslashes in ASCII art without requiring escape sequences. This keeps the art readable and maintainable.

9.7 Module: Achievements.py

The achievements.py file implements the achievement evaluation and granting system.

Key Function: `evaluate_achievements(player_name, db, mode_name, attempts, hints_used)`

- Retrieves player stats from database
- Initializes empty list for newly unlocked achievements
- Defines nested `grant(key)` function to avoid duplicates
- Checks six achievement conditions sequentially

Achievement Conditions:

- `first_blood`: `stats['wins'] == 1`
- `veteran`: `stats['games_played'] >= 25`
- `high_roller`: `stats['total_score'] >= 5000`
- `flawless`: `attempts == 1` (guessed on first try)
- `nightmare_slayer`: `mode_name.lower() == 'nightmare'`
- `no_help_needed`: `mode in ['hard', 'extreme', 'nightmare'] AND hints_used == 0`

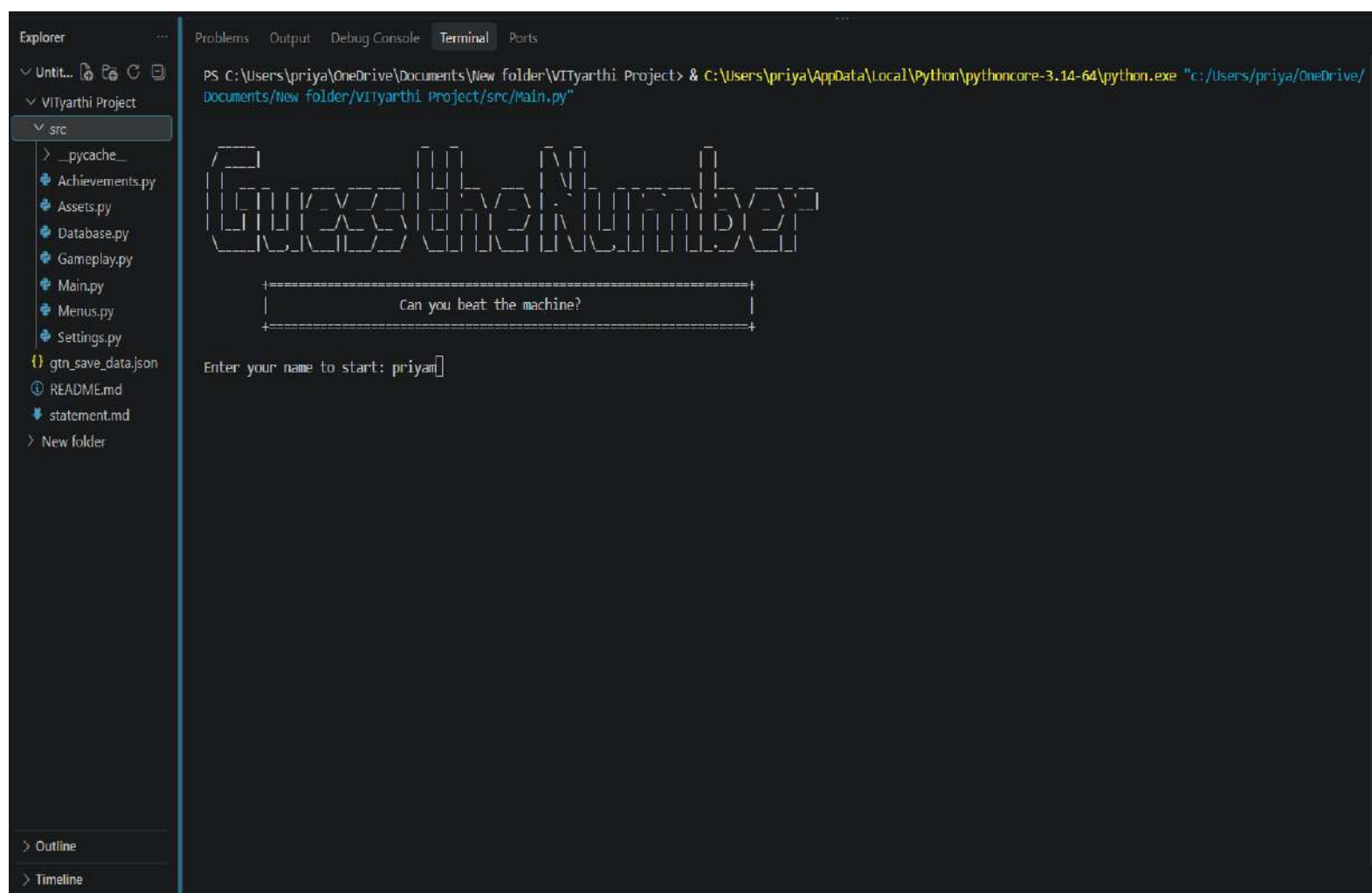
Unlock Feedback:

- Iterates through `newly_unlocked` list
- Looks up title and description from `Settings.ACHIEVEMENTS`
- Prints formatted message with emoji and achievement details
- Pauses 1.5 seconds per achievement for dramatic effect

10. SCREENSHOTS

This section contains screenshots of the application in action. Each screenshot demonstrates a key feature or screen of the game. Replace these placeholders with actual screenshots captured during gameplay.

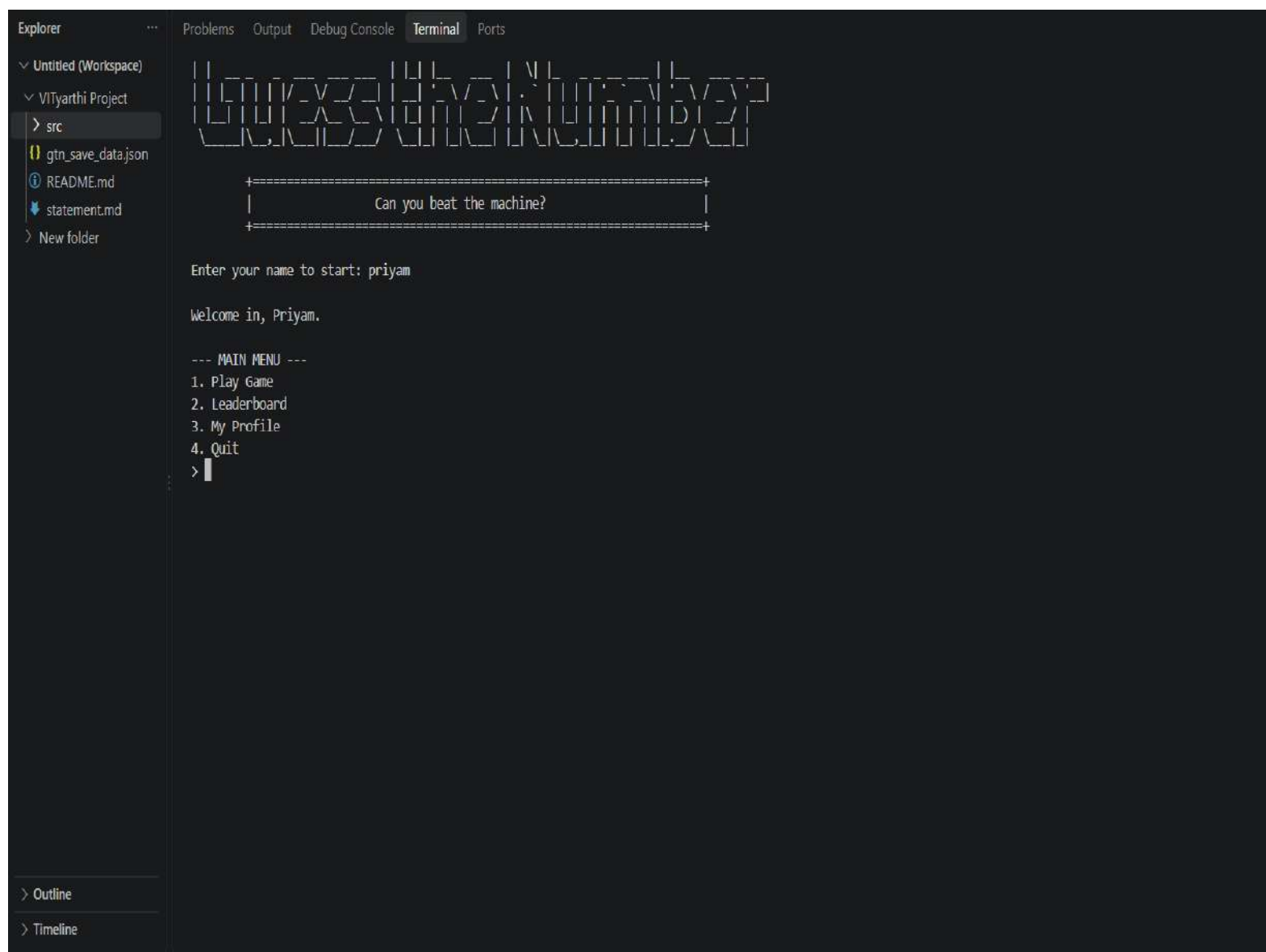
10.1 Title Screen and Player Entry



Description:

The initial screen displays the game's title in ASCII art format along with the tagline 'Can you beat the machine?'. The player is prompted to enter their name to begin. If no name is entered, the default 'Guest' is used.

10.2 Main Menu



The screenshot shows a VS Code interface with a terminal window open. The Explorer sidebar on the left shows a project named 'VITyarthi Project' with a 'src' folder containing 'gtn_save_data.json', 'README.md', and 'statement.md'. The terminal window has tabs for 'Problems', 'Output', 'Debug Console', 'Terminal' (selected), and 'Ports'. The terminal output displays a title screen with 'WELCOME TO THE GAME' in large letters, followed by a separator line and the text 'Can you beat the machine?'. Below this, it prompts the user to enter their name, which is 'priyam'. After a welcome message, it shows the 'MAIN MENU' with four options: '1. Play Game', '2. Leaderboard', '3. My Profile', and '4. Quit'. The cursor is positioned at the bottom of the menu, ready for input.

```
WELCOME TO THE GAME

=====
|               Can you beat the machine?               |
=====

Enter your name to start: priyam

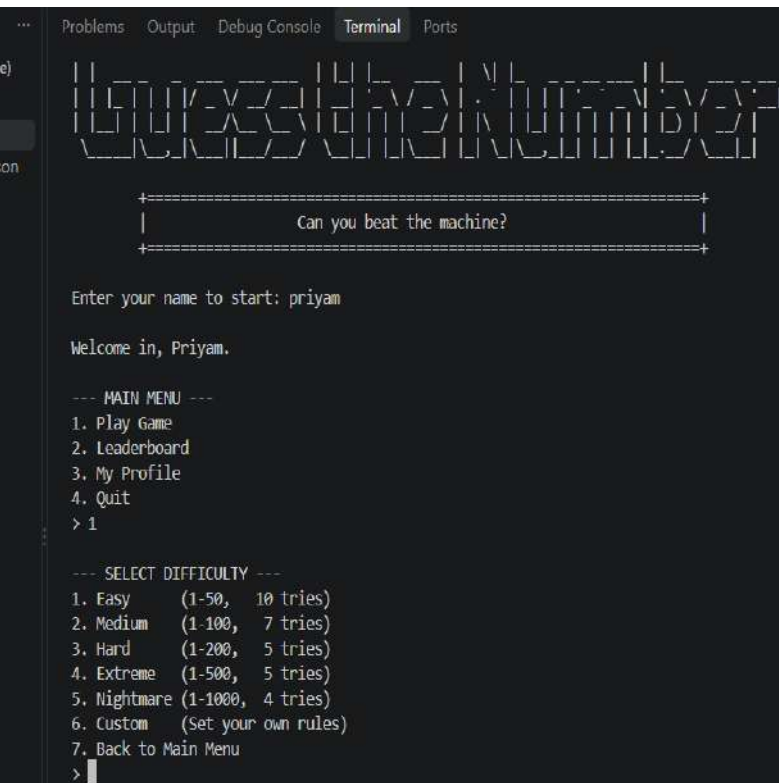
Welcome in, Priyam.

--- MAIN MENU ---
1. Play Game
2. Leaderboard
3. My Profile
4. Quit
> |
```

Description:

The main menu provides four primary navigation options. Players can start a new game, view the global leaderboard, check their personal profile, or exit the application.

10.3 Difficulty Selection



```

+-----+
|               Can you beat the machine?               |
+-----+

Enter your name to start: priyam

Welcome in, Priyam.

--- MAIN MENU ---
1. Play Game
2. Leaderboard
3. My Profile
4. Quit
> 1

--- SELECT DIFFICULTY ---
1. Easy      (1-50,  10 tries)
2. Medium    (1-100, 7 tries)
3. Hard      (1-200, 5 tries)
4. Extreme   (1-500, 5 tries)
5. Nightmare (1-1000, 4 tries)
6. Custom    (Set your own rules)
7. Back to Main Menu
> 
```

Description:

Players select their desired difficulty level. Each mode displays the number range and maximum attempts. The Custom option allows players to define their own parameters.

10.4 Gameplay - Easy Mode

```

Explorer  Problems  Output  Debug Console  Terminal  Ports
v Untitled (Workspace)
v VITYarhi Project
  > src
    gtn_save_data.json
    README.md
    statement.md
  > New folder

Enter your name to start: priyam

Welcome in, Priyam.

--- MAIN MENU ---
1. Play Game
2. Leaderboard
3. My Profile
4. Quit
> 1

--- SELECT DIFFICULTY ---
1. Easy      (1-50, 10 tries)
2. Medium    (1-100, 7 tries)
3. Hard      (1-200, 5 tries)
4. Extreme   (1-500, 5 tries)
5. Nightmare (1-1000, 4 tries)
6. Custom    (Set your own rules)
7. Back to Main Menu
> 1

      (o)  (o)
     /  \
    /    \
   /      \
  /        \
 /          \
/            \
 \          /
  \        /
   \      /
    \    /
     \  /
      (o)  (o)

--- EASY MODE ---
Target: 1 to 50.
Attempts: 10.
Need help? Type 'hint' (costs points!).

[1/10] Your guess:

```

```

Explorer  Problems  Output  Debug Console  Terminal  Ports
v Untitled (Workspace)
v VITYarhi Project
  > src
    gtn_save_data.json
    README.md
    statement.md
  > New folder

6. Custom    (Set your own rules)
7. Back to Main Menu
> 1

      (o)  (o)
     /  \
    /    \
   /      \
  /        \
 /          \
/            \
 \          /
  \        /
   \      /
    \    /
     \  /
      (o)  (o)

--- EASY MODE ---
Target: 1 to 50.
Attempts: 10.
Need help? Type 'hint' (costs points!).

[1/10] Your guess: 25
  Too high!
  Trust your instincts on this one.

History: 25
[2/10] Your guess: 12
  Too high!
  Trust your instincts on this one.

History: 25, 12
[3/10] Your guess: 6
  Too high!

History: 25, 12, 6
[4/10] Your guess: 3
  Too high!
  I've seen random number generators with better logic.

History: 25, 12, 6, 3
[5/10] Your guess: 2

```

Description:

Easy mode displays friendly ASCII art and provides a range of 1-50 with 10 attempts. Players can see their attempt counter and are prompted to enter guesses or type 'hint' for assistance.

10.5 Gameplay - Nightmare Mode

The image shows a VS Code editor interface with a dark theme. On the left, the Explorer sidebar displays a project named 'VITyarthi Project' with a 'src' directory containing files like 'gtn_save_data.json', 'README.md', and 'statement.md'. The main editor area shows a Python script for a Hangman game. The script includes a menu to select difficulty (Easy, Medium, Hard, Extreme, Nightmare, Custom) and a large ASCII art of a cat. The terminal at the bottom shows the game running in 'NIGHTMARE MODE' with a target of 1 to 1000, 4 attempts, and a prompt for a guess.

The screenshot shows a VS Code editor with a project named "VTIYarathi Project". The file explorer on the left shows files like "gtn_save_data.json", "README.md", and "statement.md". The terminal window displays the following text:

```

--- NIGHTMARE MODE ---
Target: 1 to 1000.
Attempts: 4.
Need help? Type 'hint' (costs points!).

[1/4] Your guess: 500
❌ Too high!

History: 500
[2/4] Your guess: 230
❌ Too low!

History: 500, 230
[3/4] Your guess: 339
❌ Too low!

History: 500, 230, 339
[4/4] Your guess: 432
❌ Too low!
```

Description:

Nightmare mode features intimidating dragon ASCII art and challenges players with a 1-1000 range and only 4 attempts, requiring strategic guessing and minimal reliance on hints.

10.6 Hint System in Action

```

Explorer    Problems    Output    Debug Console    Terminal    Ports
v Untitled (Workspace)
v VITyarthi Project
  > src
    {} gtn_save_data.json
    {} README.md
    {} statement.md
  > New folder

--- MAIN MENU ---
1. Play Game
2. Leaderboard
3. My Profile
4. Quit
> 1

--- SELECT DIFFICULTY ---
1. Easy      (1-50,   10 tries)
2. Medium    (1-100,  7 tries)
3. Hard      (1-200,  5 tries)
4. Extreme   (1-500,  5 tries)
5. Nightmare (1-1000, 4 tries)
6. Custom    (Set your own rules)
7. Back to Main Menu
> 1

      .--.
     (o) (o)
    /  _  \
   /  _  \
  /  _  \
 /  _  \
/  _  \

--- EASY MODE ---
Target: 1 to 50.
Attempts: 10.
Need help? Type 'hint' (costs points!).

[1/10] Your guess: hint

--- HINT 1 ---
💡 The number is ODD.
-----

[1/10] Your guess:

```

```

Explorer    Problems    Output    Debug Console    Terminal    Ports
v Untitled (Workspace)
v VITyarthi Project
  > src
    {} gtn_save_data.json
    {} README.md
    {} statement.md
  > New folder

--- SELECT DIFFICULTY ---
1. Easy      (1-50,   10 tries)
2. Medium    (1-100,  7 tries)
3. Hard      (1-200,  5 tries)
4. Extreme   (1-500,  5 tries)
5. Nightmare (1-1000, 4 tries)
6. Custom    (Set your own rules)
7. Back to Main Menu
> 1

      .--.
     (o) (o)
    /  _  \
   /  _  \
  /  _  \
 /  _  \
/  _  \

--- EASY MODE ---
Target: 1 to 50.
Attempts: 10.
Need help? Type 'hint' (costs points!).

[1/10] Your guess: hint

--- HINT 1 ---
💡 The number is ODD.
-----

[1/10] Your guess: hint

--- HINT 2 ---
💡 The number is NOT a multiple of 4.
-----

[1/10] Your guess:

```

Description:

The hint system provides two types of assistance: parity (even/odd) for the first hint, and divisibility by a random number (3, 4, 5, 7, or 10) for the second hint. Each hint incurs a 25% point penalty.

10.7 Victory Screen

```

Explorer  Problems  Output  Debug Console  Terminal  Ports
v Untitled (Workspace)
v VITyarthi Project
  > src
    gtn_save_data.json
    README.md
    statement.md
  > New folder

[1/10] Your guess: 19
Too low!
You're making this way too easy for me.

History: 19
[2/10] Your guess: 39
Too low!
Solid attempt. You're narrowing it down!

History: 19, 39
[3/10] Your guess: 47
Too high!
Just give up, I already won.

History: 19, 39, 47
[4/10] Your guess: 43

      .--.
     /  _  \
    /  (  )  \
   /  (  )  \
  /  (  )  \
 /  (  )  \
/  (  )  \
 \  (  )  /
  \  (  ) /
   \  (  )/
    \  (  )/
     \  (  )/
      --'

V W I N

Spot on! You got it in 4 tries.
Hint penalty applied: -350 pts
Score Awarded: 350 pts (Multiplier: 7x)

NEW PERSONAL BEST FOR EASY!

ACHIEVEMENT UNLOCKED: First Blood
-> Win your very first game.
Press Enter to head back...

```

```

Explorer  Problems  Output  Debug Console  Terminal  Ports
v Untitled (Workspace)
v VITyarthi Project
  > src
    gtn_save_data.json
    README.md
    statement.md
  > New folder

Trust your instincts on this one.

History: 25
[2/10] Your guess: 12
Too high!
Trust your instincts on this one.

History: 25, 12
[3/10] Your guess: 6
Too high!

History: 25, 12, 6
[4/10] Your guess: 3
Too high!
I've seen random number generators with better logic.

History: 25, 12, 6, 3
[5/10] Your guess: 2

      .--.
     /  _  \
    /  (  )  \
   /  (  )  \
  /  (  )  \
 /  (  )  \
/  (  )  \
 \  (  )  /
  \  (  ) /
   \  (  )/
    \  (  )/
     \  (  )/
      --'

V W I N

Spot on! You got it in 5 tries.
Score Awarded: 600 pts (Multiplier: 6x)

NEW PERSONAL BEST FOR EASY!

ACHIEVEMENT UNLOCKED: First Blood
-> Win your very first game.
Press Enter to head back...

```

Description:

Upon correctly guessing the number, players see a victory screen with trophy ASCII art, the number of attempts taken, calculated score with multiplier, any hint penalties applied, and notification of new personal bests.

10.8 Defeat Screen

The image shows a VS Code editor interface with a dark theme. On the left, the Explorer sidebar shows a project named 'VITyarthi Project' with a 'src' folder containing 'gtn_save_data.json', 'README.md', and 'statement.md'. The main editor area displays a terminal window with the following text:
--- NIGHTMARE MODE ---
Target: 1 to 1000.
Attempts: 4.
Need help? Type 'hint' (costs points!).
[1/4] Your guess: 500
❌ Too high!
History: 500
[2/4] Your guess: 230
❌ Too low!
History: 500, 230
[3/4] Your guess: 339
❌ Too low!
History: 500, 230, 339
[4/4] Your guess: 432
❌ Too low!

Below the text is ASCII art of a skull and the word 'GAMOVER' in a stylized, blocky font.
Out of tries, Shubham.
The number I was thinking of was 497.
Press Enter to walk away in shame...
At the bottom of the VS Code window, the Outline and Timeline panels are visible, both showing '>' icons.

Description:

When attempts are exhausted without a correct guess, players see a defeat screen with sad face ASCII art, a message indicating they're out of tries, and the revelation of what the secret number was.

10.9 Achievement Unlock

[illegible]

The image shows a VS Code editor interface with a dark theme. On the left, the Explorer sidebar shows a project named 'VITyarthi Project' with a 'src' folder containing 'gtn_save_data.json', 'README.md', and 'statement.md'. The main editor area displays a Python script for a 'Guess the Number' game. The terminal window at the bottom shows the game's execution. The user has made five guesses: 12, 6, 3, 2, and 1. The game outputs 'You Win!' in a large, stylized font. It then awards 600 points with a 6x multiplier and unlocks an achievement called 'First Blood' for winning the first game. The terminal text is as follows:

```
History: 25  
[2/10] Your guess: 12  
Too high!  
Trust your instincts on this one.  
  
History: 25, 12  
[3/10] Your guess: 6  
Too high!  
  
History: 25, 12, 6  
[4/10] Your guess: 3  
Too high!  
I've seen random number generators with better logic.  
  
History: 25, 12, 6, 3  
[5/10] Your guess: 2  
  
You Win!  
  
Spot on! You got it in 5 tries.  
Score Awarded: 600 pts (Multiplier: 6x)  
  
NEW PERSONAL BEST FOR EASY!  
  
ACHIEVEMENT UNLOCKED: First Blood  
-> Win your very first game.  
Press Enter to head back...
```

Description:

After a victory, the game checks achievement conditions. When met, players see a celebratory notification with the achievement title and description, paused for 1.5 seconds for dramatic effect.

10.10 Global Leaderboard

```
Problems Output Debug Console Terminal Ports
```

Press Enter to head back...

```
--- MAIN MENU ---  
1. Play Game  
2. Leaderboard  
3. My Profile  
4. Quit  
> 7  
Nope, type 1, 2, 3, or 4.  
  
--- MAIN MENU ---  
1. Play Game  
2. Leaderboard  
3. My Profile  
4. Quit  
> 2
```



```
      _==_
    .-:.   -.-.
 | (:.. )| 
 |-|.     ||
 -.|.:   |
 \::.. / 
 :.../    
 )(      
 |__|_____|
 [_____]
```

```
--- GLOBAL LEADERBOARD ---  
Rank | Player           | Score | Wins | Win %  
-----  
#1   | Shubham          | 600   | 1    | 50.0%  
#2   | Priyam           | 350   | 1    | 50.0%  
#3   | & c:\users\p...  | 0     | 0    | 0.0%
```

Press Enter to go back...

Description:

The leaderboard displays all players ranked by total score in descending order. Each entry shows rank, player name, total accumulated score, total wins, and win percentage calculated from games played.

10.11 Player Profile - Stats View

```

Explorer    Problems  Output  Debug Console  Terminal  Ports
v Untitled (Workspace)
v VITyarthi Project
  > src
    gtn_save_data.json
    README.md
    statement.md
  > New folder

--- GLOBAL LEADERBOARD ---
Rank | Player      | Score | Wins | Win %
-----|-----|-----|-----|-----
#1   | Shubham     | 600   | 1    | 50.0%
#2   | Priyam      | 350   | 1    | 50.0%
#3   | & c:\users\p... | 0     | 0    | 0.0%

Press Enter to go back...

--- MAIN MENU ---
1. Play Game
2. Leaderboard
3. My Profile
4. Quit
> 3

[ \---/ ]  PLAYER PROFILE
> . . <
[ "----" ]
^-----^

--- PROFILE: SHUBHAM ---
1. View My Stats
2. View My Badges/Achievements
3. Switch Player
4. Back to Main Menu
>

```

```

Explorer    Problems  Output  Debug Console  Terminal  Ports
v Untitled (Workspace)
v VITyarthi Project
  > src
    gtn_save_data.json
    README.md
    statement.md
  > New folder

1. Play Game
2. Leaderboard
3. My Profile
4. Quit
> 3

[ \---/ ]  PLAYER PROFILE
> . . <
[ "----" ]
^-----^

--- PROFILE: SHUBHAM ---
1. View My Stats
2. View My Badges/Achievements
3. Switch Player
4. Back to Main Menu
> 1

--- Shubham's Lifetime Stats ---
Total Points: 600
Games Played: 2
Total Wins: 1
Win Rate: 50.0%

-- Personal Bests --
Easy      : 5
Medium    : Unbeaten
Hard      : Unbeaten
Extreme   : Unbeaten
Nightmare: Unbeaten

Press Enter...

```

Description:

The profile stats view shows comprehensive player statistics including lifetime points, games played, wins, calculated win rate, and personal best attempt counts for each difficulty mode (or 'Unbeaten' for modes not yet completed).

10.12 Player Profile - Achievements View

```

Explorer    Problems    Output    Debug Console    Terminal    Ports
v Untitled (Workspace)
v VITyarthi Project
  > src
  gtn_save_data.json
  README.md
  statement.md
  > New folder

--- Shubham's Lifetime Stats ---
Total Points: 600
Games Played: 2
Total Wins: 1
Win Rate: 50.0%

-- Personal Bests --
Easy : 5
Medium : Unbeaten
Hard : Unbeaten
Extreme : Unbeaten
Nightmare : Unbeaten

Press Enter...

( \---/ )  PLAYER PROFILE
> . . <
( "----" )

--- PROFILE: SHUBHAM ---
1. View My Stats
2. View My Badges/Achievements
3. Switch Player
4. Back to Main Menu
> 2

--- Shubham's Trophy Case ---
🏆 First Blood: Win your very first game.

Press Enter...

```

```

Explorer    Problems    Output    Debug Console    Terminal    Ports
v Untitled (Workspace)
v VITyarthi Project
  > src
  gtn_save_data.json
  README.md
  statement.md
  > New folder

1. View My Stats
2. View My Badges/Achievements
3. Switch Player
4. Back to Main Menu
> 3

Enter new player name: priyam
Swapped to Priyam.

--- MAIN MENU ---
1. Play Game
2. Leaderboard
3. My Profile
4. Quit
> 3

( \---/ )  PLAYER PROFILE
> . . <
( "----" )

--- PROFILE: PRIYAM ---
1. View My Stats
2. View My Badges/Achievements
3. Switch Player
4. Back to Main Menu
> 2

--- Priyam's Trophy Case ---
🏆 First Blood: Win your very first game.

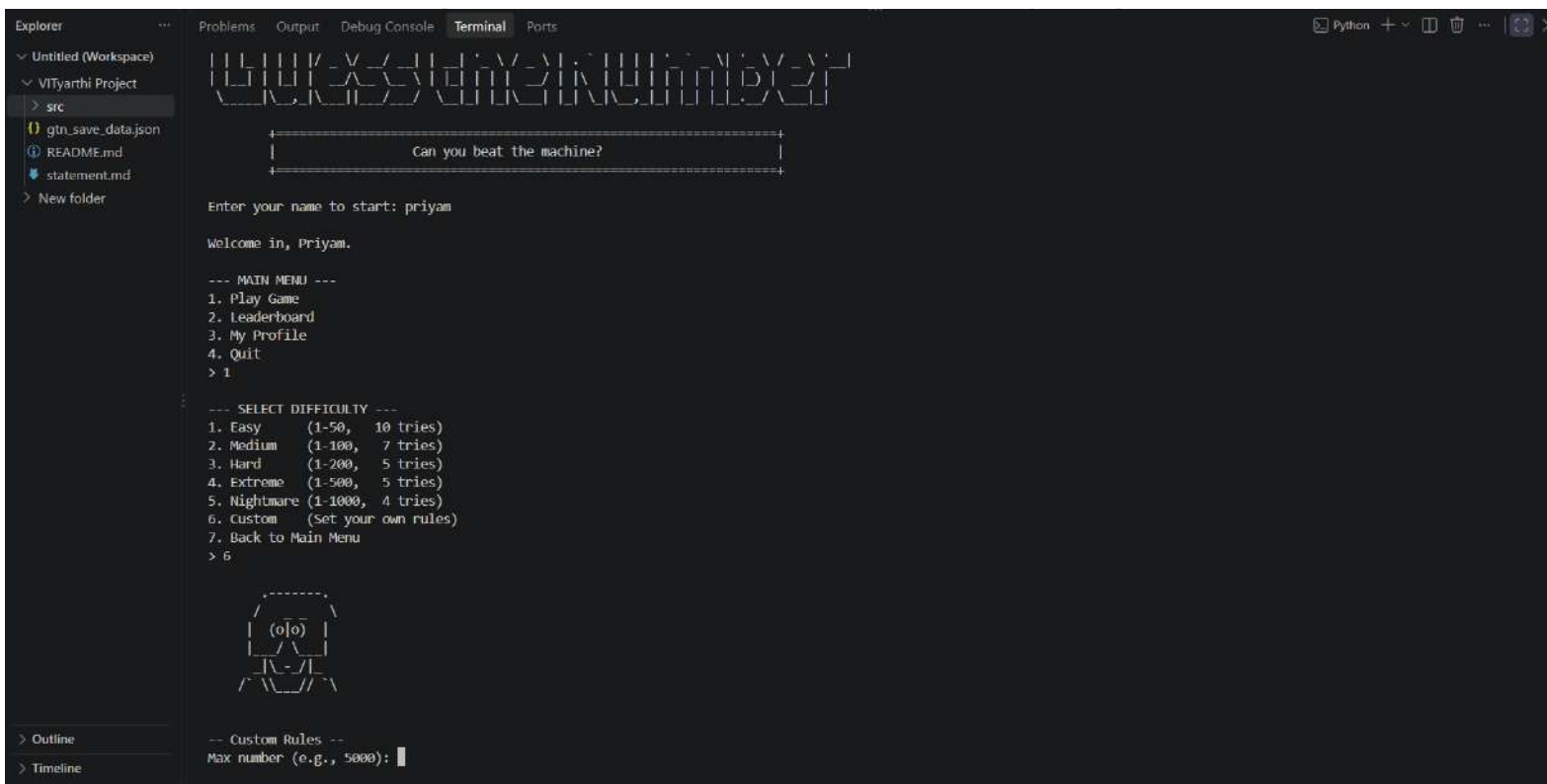
Press Enter...

```

Description:

The achievements view displays all badges the player has unlocked, each with a trophy emoji, the achievement title, and its description. Empty trophy cases encourage continued play.

10.13 Custom Game Mode Setup



```

Explorer  ...  Problems  Output  Debug Console  Terminal  Ports
v Untitled (Workspace)
v VITyarthi Project
  > src
    gtn_save_data.json
    README.md
    statement.md
  > New folder

Can you beat the machine?

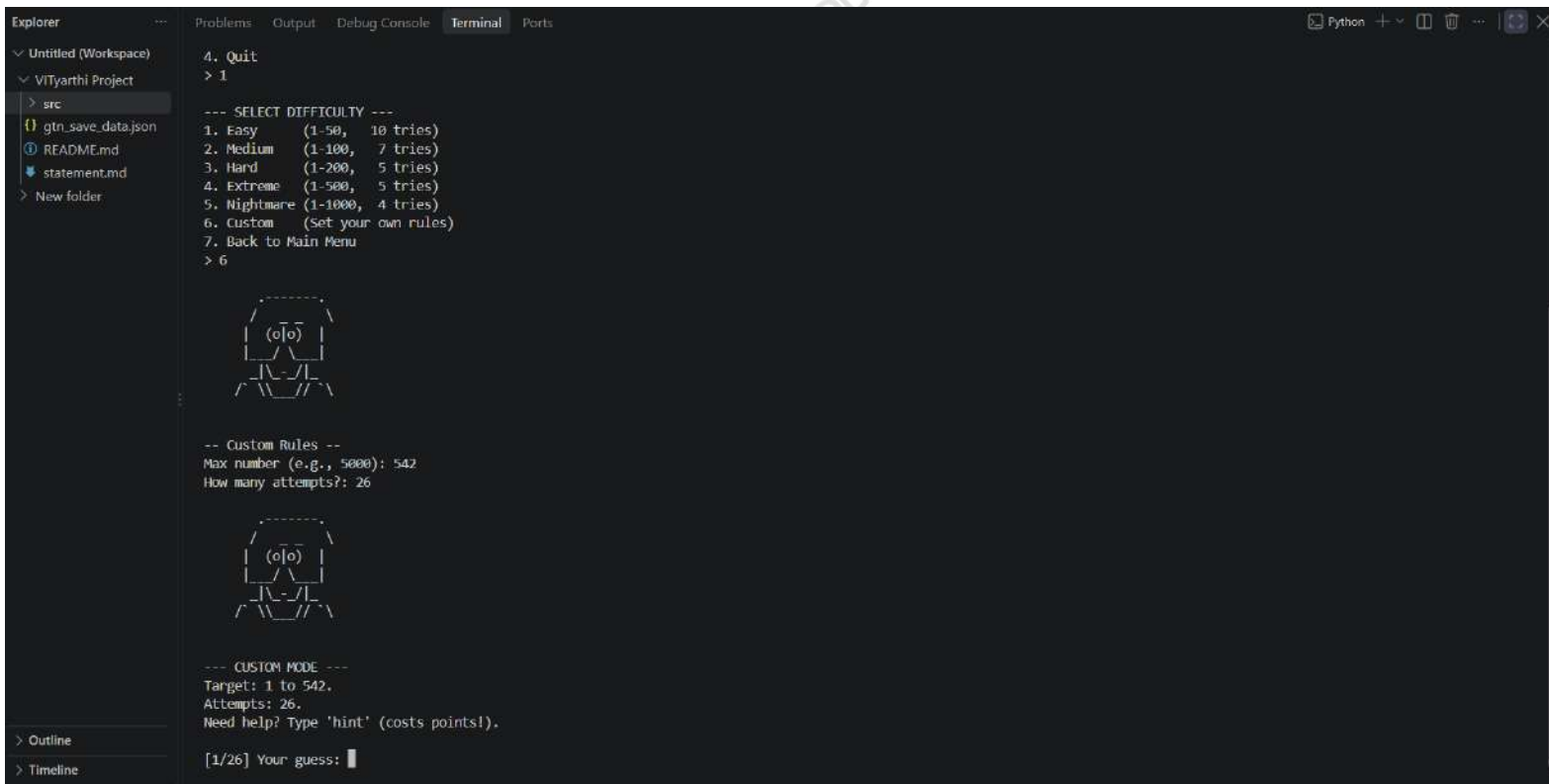
Enter your name to start: priyam

Welcome in, Priyam.

--- MAIN MENU ---
1. Play Game
2. Leaderboard
3. My Profile
4. Quit
> 1

--- SELECT DIFFICULTY ---
1. Easy (1-50, 10 tries)
2. Medium (1-100, 7 tries)
3. Hard (1-200, 5 tries)
4. Extreme (1-500, 5 tries)
5. Nightmare (1-1000, 4 tries)
6. Custom (Set your own rules)
7. Back to Main Menu
> 6

-- Custom Rules --
Max number (e.g., 5000):
```



```

Explorer  ...  Problems  Output  Debug Console  Terminal  Ports
v Untitled (Workspace)
v VITyarthi Project
  > src
    gtn_save_data.json
    README.md
    statement.md
  > New folder

4. Quit
> 1

--- SELECT DIFFICULTY ---
1. Easy (1-50, 10 tries)
2. Medium (1-100, 7 tries)
3. Hard (1-200, 5 tries)
4. Extreme (1-500, 5 tries)
5. Nightmare (1-1000, 4 tries)
6. Custom (Set your own rules)
7. Back to Main Menu
> 6

-- Custom Rules --
Max number (e.g., 5000): 542
How many attempts?: 26

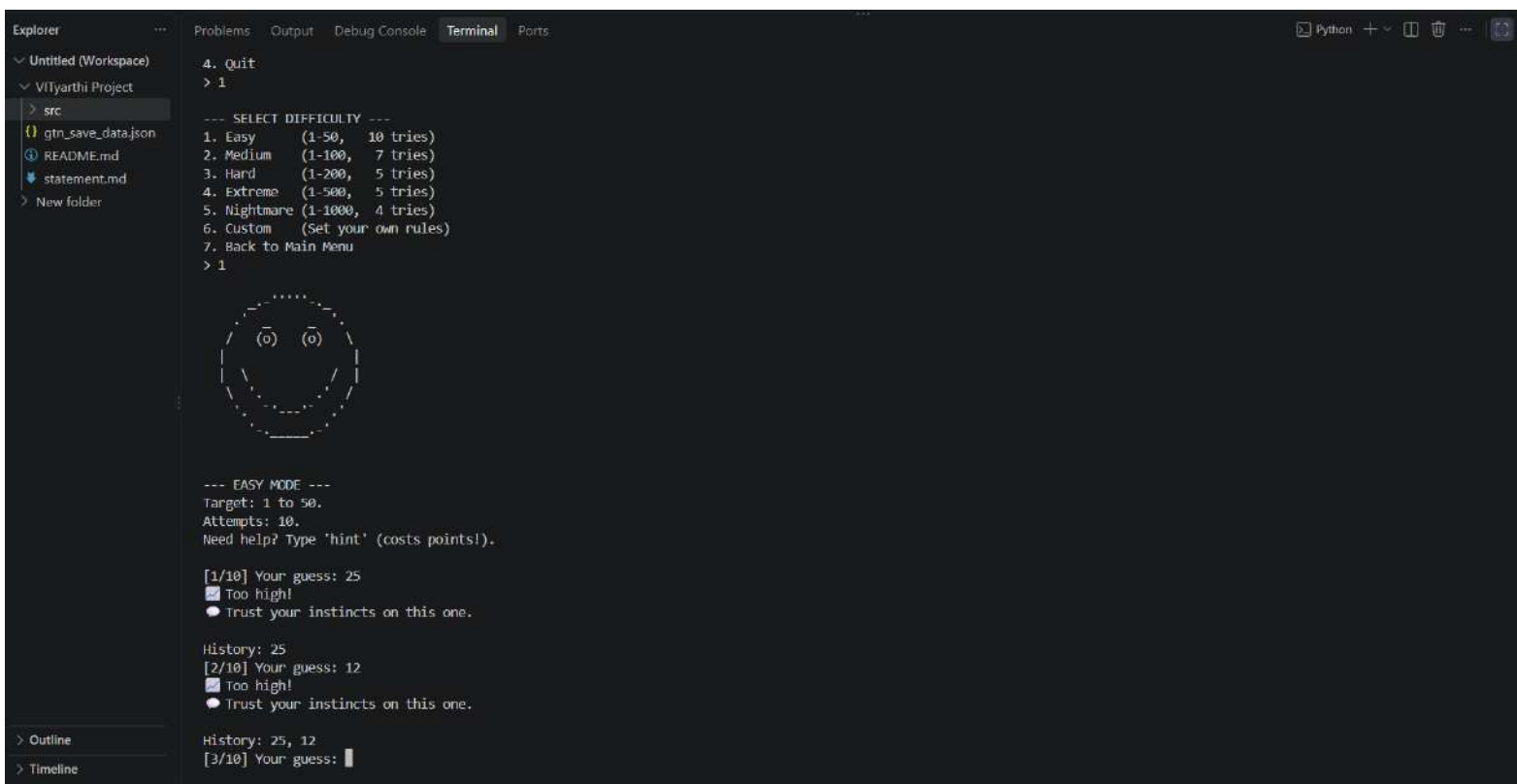
--- CUSTOM MODE ---
Target: 1 to 542.
Attempts: 26.
Need help? Type 'hint' (costs points!).

[1/26] Your guess:
```

Description:

Custom mode allows players to define their own challenge parameters. The game prompts for maximum number and attempt count, validates that numbers are reasonable (high ≥ 2 , tries ≥ 1), and then launches the custom game.

10.14 Motivational/Demotivational Feedback



```
4. Quit
> 1

--- SELECT DIFFICULTY ---
1. Easy      (1-50, 10 tries)
2. Medium    (1-100, 7 tries)
3. Hard      (1-200, 5 tries)
4. Extreme   (1-500, 5 tries)
5. Nightmare (1-1000, 4 tries)
6. Custom    (Set your own rules)
7. Back to Main Menu
> 1

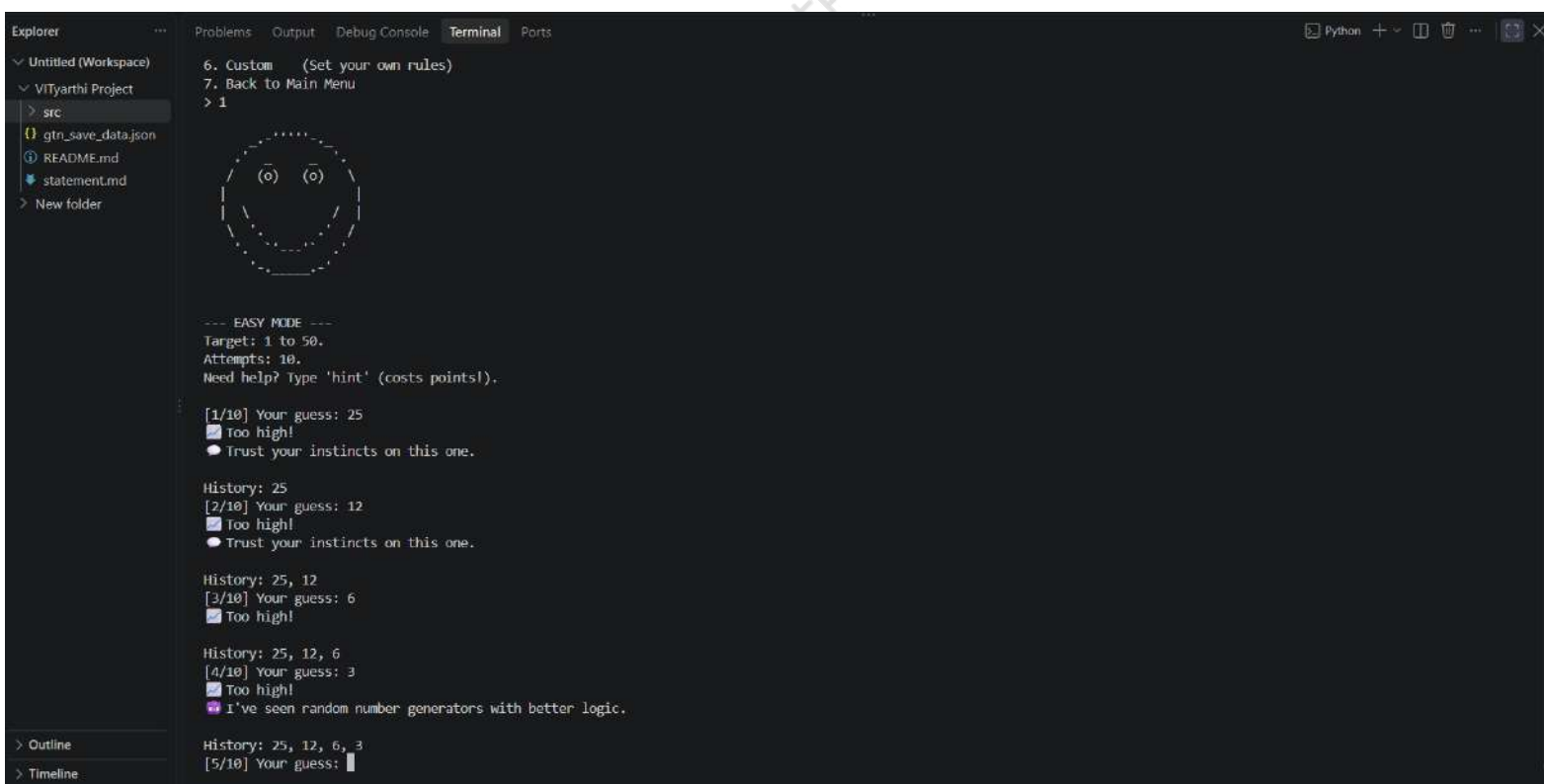
  (o)  (o)
  /  \
 /____\
|       |
|       |
|       |
|_____|

--- EASY MODE ---
Target: 1 to 50.
Attempts: 10.
Need help? Type 'hint' (costs points!).

[1/10] Your guess: 25
❑ Too high!
● Trust your instincts on this one.

History: 25
[2/10] Your guess: 12
❑ Too high!
● Trust your instincts on this one.

History: 25, 12
[3/10] Your guess: 
```



```
6. Custom    (Set your own rules)
7. Back to Main Menu
> 1

  (o)  (o)
  /  \
 /____\
|       |
|       |
|       |
|_____|

--- EASY MODE ---
Target: 1 to 50.
Attempts: 10.
Need help? Type 'hint' (costs points!).

[1/10] Your guess: 25
❑ Too high!
● Trust your instincts on this one.

History: 25
[2/10] Your guess: 12
❑ Too high!
● Trust your instincts on this one.

History: 25, 12
[3/10] Your guess: 6
❑ Too high!

History: 25, 12, 6
[4/10] Your guess: 3
❑ Too high!
💜 I've seen random number generators with better logic.

History: 25, 12, 6, 3
[5/10] Your guess: 
```

Description:

After incorrect guesses, the game has a 35% chance to display a random message. Motivational messages encourage persistence, while demotivational messages provide comic relief without actual penalties.

10.15 Error Handling - Invalid Input

[illegible]

The image shows a VS Code editor window with a Python project named 'Vitiyarthi Project'. The Explorer sidebar on the left shows the project structure, including a 'src' folder, 'gtn_save_data.json', 'README.md', and 'statement.md'. The main editor area displays the game's code, which includes a main menu, difficulty selection, and a medium mode game loop. The game logic involves guessing a target number (1-100) within 7 attempts, with a hint system. The current state shows the user has entered 'b' as a guess, and the program prompts them to try again. The terminal output shows the game's progress and the user's input.

Description:

The game gracefully handles invalid input including non-numeric guesses, duplicate guesses, and out-of-range entries. Error messages are friendly and informative, guiding players toward valid input.

10.16 Quit Screen

[SCREENSHOT PLACEHOLDER 22: Quit Screen]

Description:

When players choose to quit, they see a tombstone ASCII art with 'RIP GAME OVER' and a personalized farewell message. The same screen appears on Ctrl+C interrupt for consistent exit behavior.